Testnet Portal — Design & Implementation

> ***Purpose:*** *A dedicated testnet portal where participants register as* ***developer****,* ***user****, or* ***node operator****, with role-specific dashboards, community pages, and metrics. Replaces the standalone quests page with a unified experience.*
> ***References:*** *INCENTIVIZED-TESTNET.md, DEVELOPMENT-AND-ENHANCEMENTS.md, BOING-BLOCKCHAIN-DESIGN-PLAN.md.*

---

1. Overview

The **Testnet Portal** is the single place to:

- **Sign up** — Register with a testnet account ID and choose role: **Developer**, **User**, or **Node operator**.
- **Track metrics** — Each role has a dedicated dashboard and community page with relevant metrics.
- **Qualify for rewards** — Registration and activity feed into the incentivized testnet rules (Community & Grants pool). Developers have a dedicated portion of the ecosystem pool for **successful dApps** (mainnet); testnet dev participation qualifies for grants and recognition.

Site structure:

Path	Purpose
`/testnet`	Portal landing: choose role, register, or go to dashboard
`/testnet/register`	Registration form (account ID + role + optional contact)
`/testnet/developers`	Developers community + dashboard (dApps, metrics, link to dApp incentive pool)
`/testnet/users`	Users community + dashboard (quests, faucet, feedback)
`/testnet/operators`	Node operators community + dashboard (leaderboard, uptime, blocks)

The existing **Join Testnet** hub stays at `/network/testnet` (bootnodes, faucet, single-vs-multi). It links prominently to the portal for registration and dashboards. The former **Community Quests** page is folded into **Users** at `/testnet/users`.

---

2. Registration

- **Required:** Account ID (32-byte hex), Role (developer | user | node_operator).
- **Optional:** Email, Discord handle, GitHub username (for devs), node multiaddr (for operators).
- **Storage:** D1 table `portal_registrations`. One registration per account ID; role can be updated (e.g. user → developer) until testnet end.
- **API:** `POST /api/portal/register` — validate account_id and role; insert or update; return success. Rate limit per IP.
- **Verification:** No email verification required for Phase 1. Optional: later, link registration to on-chain identity (e.g. sign a message with the account key).

---

3. Role-Specific Dashboards & Metrics

3.1 Developers

- **Community page:** `/testnet/developers` — Who's building, links to docs, SDK, CLI, and **success-based dApp incentives**.

- **Dashboard (per account):**
  - **Registered dApps** — List of contracts registered for incentive tracking (via `boing_registerDappMetrics` or portal form during testnet). Fields: contract address, owner (account ID), name, registered_at.
  - **Metrics placeholder** — Fees generated, tx count (when indexer/RPC supports it); link to mainnet "dedicated portion of initial supply / ecosystem pool for successful dApps" (see BOING-BLOCKCHAIN-DESIGN-PLAN.md § Success-Based dApp Incentives, dApp cap).
- **Rewards:** Testnet BOING, bug bounties, dApp recognition; **mainnet:** eligibility for grants and for the **dApp incentive pool** (success-based, value cap per dApp). A portion of the **Community & Grants** allocation is reserved for successful dApps; testnet participation qualifies developers for that ecosystem.

Data: `portal_dapps` table (contract_hex, owner_account_hex, name, registered_at) for testnet-registered dApps when RPC `boing_registerDappMetrics` is not yet used; once used, dashboard can show data from chain + indexer.

3.2 Users

- **Community page:** `/testnet/users` — Quests, faucet link, feedback, and "how to qualify" for user rewards.
- **Dashboard (per account):**
  - **Quest progress** — List of quests (see §10 Community Quests below) with completed / pending / not started. Uses `quest_completions` + optional on-chain verification.
  - **Faucet / first tx** — Simple "used faucet" / "first tx sent" badges when verified.
- **Rewards:** Faucet + quests + feedback; optional capped NFT/mainnet recognition per Incentivized Testnet Rules.

3.3 Node operators

- **Community page:** `/testnet/operators` — How to run a node, bootnodes, VibeMiner, and leaderboard.
- **Dashboard (per account):**
  - **Validator metrics** — Blocks proposed, uptime (when available from indexer or RPC). Placeholder until testnet indexer exists.
  - **Leaderboard** — Top N by blocks or uptime; link to explorer when available.
- **Rewards:** Testnet BOING (blocks) + leaderboard/uptime; mainnet allocation or NFT for top N (capped).

---

4. Metrics Summary (What We Track)

Role	Metrics	Source
Developer	Registered dApps, (later) fees/tx per dApp	D1 `portal_dapps`; RPC/indexer
User	Quest completions, faucet use, first tx	D1 `quest_completions`; RPC for on-chain checks
Node operator	Blocks proposed, uptime, node ID	Indexer/RPC when available; D1 cache for leaderboard

---

5. Data Model (D1)

```sql
-- Portal: one row per account, role can be updated
CREATE TABLE IF NOT EXISTS portal_registrations (
  account_id_hex TEXT PRIMARY KEY,
  role TEXT NOT NULL,  -- 'developer' | 'user' | 'node_operator'
  email TEXT,
  discord_handle TEXT,
  github_username TEXT,
  node_multiaddr TEXT,
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL
);

-- Developer: dApps registered for testnet (optional; mainnet uses
boing_registerDappMetrics)
CREATE TABLE IF NOT EXISTS portal_dapps (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  contract_hex TEXT NOT NULL,
  owner_account_hex TEXT NOT NULL,
  name TEXT,
  registered_at TEXT NOT NULL,
  UNIQUE(contract_hex)
);

CREATE INDEX IF NOT EXISTS idx_portal_dapps_owner ON portal_dapps(owner_account_hex);
```

`quest_completions` and `quests` are defined in §10 (Community Quests) below; they are used by the **Users** dashboard.

---

6. API (Cloudflare Worker)

Method	Path	Purpose
POST	`/api/portal/register`	Register or update role: `{ account_id_hex, role, email?, discord_handle?, github_username?, node_multiaddr? }`
GET	`/api/portal/me?account_id_hex=0x...`	Return registration + role-specific summary (quest count, dApp count, or placeholder operator stats)
GET	`/api/portal/leaderboard?role=node_operator&limit=20`	Leaderboard for operators (when we have blocks/uptime data)
POST	`/api/portal/dapps`	Register a dApp (dev only): `{ contract_hex, owner_account_hex, name? }`
GET	`/api/quests/status?account_id_hex=0x...`	Quest progress for users (reuse or extend existing quest API)

---

7. Implementation Phases

Phase	Scope

Phase 0	Static portal: `/testnet` landing with role cards and link to `/testnet/register`; `/testnet/developers`, `/testnet/users`, `/testnet/operators` as static community pages with dashboard placeholders and links to faucet/quests/docs. Registration form POSTs to API or external form.
Phase 1	D1 + API: `portal_registrations`, `portal_dapps`; implement register and me endpoints; dashboards show "Registered as X" and (for users) link to quests; (for devs) list of registered dApps from form.
Phase 2	Quest integration: Users dashboard shows quest list and progress from `quest_completions`; on-chain verification for faucet/first_tx.
Phase 3	Operator metrics: Ingest blocks/proposer from indexer or RPC; leaderboard and per-account blocks/uptime on operators dashboard.

---

8. dApp Incentive Pool (Context for Developers)

From [BOING-BLOCKCHAIN-DESIGN-PLAN.md]() and [DEVELOPMENT-AND-ENHANCEMENTS.md]():

- **Success-based dApp incentives** — Rewards are based on usage (e.g. fees generated by the dApp's contracts), with a **per-dApp value cap** per epoch.
- **Total dApp pool** — A portion of fees (e.g. % of treasury share) and/or of the **Community & Grants** allocation is reserved for successful dApps. This acts as the "dedicated portion" for builders; testnet participation qualifies developers for grants and for inclusion in this ecosystem at mainnet.
- **Transparent reporting** — Dashboard and SDK for dApp owners to track earned incentives (mainnet). Testnet portal gives early visibility and registration.

The portal's **Developers** page and dashboard should state this clearly and link to the design doc and to the Incentivized Testnet Rules.

---

9. Checklist

- [x] Add `portal_registrations` and `portal_dapps` to schema; apply migration.
- [x] Implement `/testnet` (landing), `/testnet/register`, `/testnet/developers`, `/testnet/users`, `/testnet/operators` (static or with API).
- [x] Implement `POST /api/portal/register` and `GET /api/portal/me` (and optional `POST /api/portal/dapps`; user quest progress via portal/me).
- [x] From `/network/testnet`, add prominent "Testnet Portal → Register & dashboards" link; from quests, redirect or link to `/testnet/users`.
- [x] Document portal in INCENTIVIZED-TESTNET.md and in this doc §10 (Community Quests; quests live under Users in the portal).

---

10. Community Quests

Community quests are user-facing tasks (e.g. "Use the faucet", "Send a transaction", "Share feedback") that qualify participants for testnet rewards. Completion can be **auto-verifiable on-chain** (e.g. faucet receipt, first tx via RPC/D1 indexer) or **manual with proof** (user submits account ID + proof; team verifies later). Publish a single **Quests** page listing tasks, verification type, and reward eligibility. Collect submissions via a form (website form → API → D1, or external form + spreadsheet) and process them before testnet end.

10.1 Quest types and examples

Quest ID	Name	Description	Verification	Reward tier
`faucet`	First drip	Request testnet BOING from the faucet	On-chain: faucet tx or balance > 0	Base user
`first_tx`	First transaction	Send any transaction on testnet	On-chain: account has nonce ≥ 1 or tx in block	Base user
`validator_connect`	Join the network	Run a node connected to testnet bootnodes	Manual: submit node ID / multiaddr or screenshot	Validator track
`feedback`	Share feedback	Answer 3–5 short questions (UX, docs, bugs)	Manual: form submission + optional account ID	Bonus
`social`	Join community	Join Discord and post in #testnet-intros	Manual: Discord handle + account ID	Bonus
`docs`	Read and confirm	Visit Getting Started + Testnet docs, confirm checkbox	Manual: form with "I have read" + account ID	Base user

Phase 1 recommendation: Start with 3–5 quests: `faucet`, `first_tx`, `feedback`, and optionally `social` and `validator_connect`.

10.2 Verification methods

On-chain (auto): After user requests from faucet, call testnet RPC to check balance or index faucet txs in D1. For first tx: RPC `eth_getTransactionCount(account_id, "latest")` > 0 or D1 query for any tx where `from = account_id`. Use a Cloudflare Worker (or cron) that reads pending submissions from D1 where `verification_type = 'on_chain'` and `verified_at IS NULL`, calls testnet RPC or D1, and updates `verified_at` and `proof_value`.

Manual (proof submitted): User submits account ID (32-byte hex) plus proof (tx hash, Discord username, link, or form answers). Team reviews and sets `verified_at` and optionally `rejected_reason`. Store in D1: `quest_id`, `account_id`, `proof_type`, `proof_value`, `submitted_at`, `verified_at`, `rejected_reason`.

10.3 D1 tables (quests)

```
CREATE TABLE IF NOT EXISTS quests (
  id TEXT PRIMARY KEY,
  name TEXT NOT NULL,
  description TEXT,
  verification_type TEXT NOT NULL,  -- 'on_chain' | 'manual'
  reward_tier TEXT,                 -- 'base' | 'validator' | 'bonus'
  active INTEGER DEFAULT 1,
  created_at TEXT
);

CREATE TABLE IF NOT EXISTS quest_completions (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
  quest_id TEXT NOT NULL,
  account_id_hex TEXT NOT NULL,
  proof_type TEXT,
  proof_value TEXT,
  submitted_at TEXT NOT NULL,
  verified_at TEXT,
  rejected_reason TEXT,
  FOREIGN KEY (quest_id) REFERENCES quests(id)
);
CREATE INDEX IF NOT EXISTS idx_quest_completions_account ON
quest_completions(account_id_hex);
CREATE INDEX IF NOT EXISTS idx_quest_completions_quest ON quest_completions(quest_id);
CREATE UNIQUE INDEX IF NOT EXISTS idx_quest_completions_unique ON
quest_completions(quest_id, account_id_hex);
```

10.4 Quest API

Method	Path	Purpose
GET	`/api/quests`	List active quests (id, name, description, verification_type, reward_tier).
POST	`/api/quests/submit`	Submit a completion: `{ "quest_id", "account_id_hex", "proof_type?", "proof_value?" }`. Validate account_id format; insert into `quest_completions`; return `{ "ok": true, "id" }`. Rate limit per IP/account.
GET	`/api/quests/status?account_id_hex=0x...`	Return list of quest completions for that account (for "My progress" UI).

10.5 Incentive rules and implementation checklist

Define in a single **Incentivized Testnet Rules** page (e.g. `/network/incentivized-rules`): which quests are live and their reward tier; cap per user; that quest completion qualifies for Community & Grants pool rewards. Link from the Quests page and announcements.

- **Phase 0 (no backend):** Static `/network/quests` page; define 3–5 quests in copy; link to external form for manual submissions.
- **Phase 1 (backend):** Add D1 tables `quests`, `quest_completions`; seed `quests`; implement GET `/api/quests` and POST `/api/quests/submit`; add form on Quests page; optional GET `/api/quests/status`.
- **Phase 2 (auto-verify):** Implement on-chain verification (scheduled or on-demand) for `faucet` and `first_tx`; update `verified_at` in D1.
- **Launch:** Publish Incentivized Testnet Rules; link Quests from testnet hub and announcements; at testnet end, export completions and distribute rewards per rules.

Boing Network — Authentic. Decentralized. Optimal. Sustainable.